

Perfect 🎯 — this one is also widely used in **Snowflake data masking** to partially hide sensitive information such as **phone numbers**, **credit cards**, or **account numbers**.

Let's break it down step by step 📖

❏ **Function:** `CONCAT('XXXXXX', RIGHT(val, 4))`

This expression combines two functions:

- `CONCAT()` — joins (concatenates) strings.
 - `RIGHT()` — extracts characters from the right end of a string.
-

⚙️ Step-by-Step Breakdown

Let's say:

```
val = '9876543210'
```

❏ **1** `RIGHT(val, 4)`

This takes the **last 4 characters** of the string.

```
SELECT RIGHT('9876543210', 4);
```

✓ **Result:** '3210'

❏ **2** `CONCAT('XXXXXX', RIGHT(val, 4))`

This concatenates 'XXXXXX' (six Xs) with the last four digits.

```
SELECT CONCAT('XXXXXX', RIGHT('9876543210', 4));
```

✓ **Result:** 'XXXXXX3210'

✓ **Purpose:**

This creates a **masked version** of the phone number where only the last 4 digits are visible.

Example Usage:

```
CREATE MASKING POLICY phone_mask
AS (val STRING) RETURNS STRING ->
CASE
  WHEN CURRENT_ROLE() IN ('FULL_ACCESS_ROLE') THEN val
  ELSE CONCAT('XXXXXX', RIGHT(val, 4))
END;
```

Output:

Role	Actual Value	Masked Value
FULL_ACCESS_ROLE	9876543210	9876543210
LIMITED_ACCESS_ROLE	9876543210	XXXXXX3210

 Function Details

Function	Description	Example	Output
RIGHT(<string>, n)	Returns the last <i>n</i> characters of a string.	RIGHT('Snowflake', 4)	'flake'
CONCAT(str1, str2)	Joins two or more strings together.	CONCAT('Data', 'Base')	'DataBase'
CONCAT('XXXXXX', RIGHT('9876543210', 4))	Combines a mask with the last 4 digits.	'XXXXXX3210'	

 Common Masking Use Cases

Data Type	Expression	Result
Phone Number	CONCAT('XXXXXX', RIGHT(phone, 4))	XXXXXX3210
Credit Card	CONCAT('XXXX-XXXX-XXXX-', RIGHT(card_no, 4))	XXXX-XXXX-XXXX-1234
Account Number	CONCAT('XXXXXX', RIGHT(account_no, 3))	XXXXXX789
Email (partial)	CONCAT('xxxx@', SPLIT_PART(email, '@', 2))	xxxx@gmail.com

 Notes

- `RIGHT()` only works with **string data types** — if your column is numeric, you should cast it:
 - `RIGHT( TO_VARCHAR(phone), 4)`
 - You can adjust the number of 'x's or digits shown depending on data policy requirements.
-

✓ Summary

Expression	Meaning	Example Input	Output
<code>RIGHT(val, 4)</code>	Get last 4 characters	9876543210	3210
<code>CONCAT('XXXXXX', RIGHT(val, 4))</code>	Mask all but last 4 digits	9876543210	XXXXXX3210

Would you like me to show **a few more examples of similar masking logic** (like masking Aadhaar numbers, credit cards, or emails) that you can reuse in your Snowflake project or interview notes?

Absolutely! Let's break down **data masking** vs **encryption** in very detailed terms, so you understand not just the definitions but their purposes, mechanisms, and use cases.

1. Data Masking

Definition

Data masking is a **technique that hides sensitive information by obscuring or replacing it with fictitious, but realistic values**. The key is: **the original data remains unchanged in storage for authorized users**, but unauthorized users see only masked values.

Key Points

Aspect	Explanation
Purpose	To protect sensitive data during use, testing, or display without altering the original data physically .
Visibility	Data is hidden visually . Masked users cannot see the real value, but the real data exists in the system.
Reversibility	Usually irreversible , depending on masking type. Some dynamic masking can be reversible for authorized users.
Example	- Original SSN: 123-45-6789- Masked: xxx-xx-6789- Original credit card: 4111-1111-1111-1234- Masked: xxxx-xxxx-xxxx-1234
Implementation	Often done dynamically at query time (dynamic masking) or statically by replacing data in test/dev environments (static masking).
Use Cases	- Testing & development environments- Displaying limited info in UI (e.g., show last 4 digits)- Compliance with GDPR, HIPAA, PCI DSS

Types of Masking

1. Static Masking

- Data in databases is replaced permanently in a copy (usually for testing).
- Original production data stays safe.
- Example: Replace all real names with fake names in a test DB.

2. Dynamic Masking

- Data stays in the database, but **queries return masked data** for unauthorized users.
 - Original data remains intact for authorized roles.
 - Example: SQL Server's Dynamic Data Masking (`MASKED WITH (...)`).
-

Analogy

Think of **masking** like **putting a curtain on a window**: people outside cannot see inside clearly, but the room itself is untouched.

2. Encryption

Definition

Encryption is a **mathematical process that converts data into a coded format** so that **only authorized parties with a key can decrypt and read it**. It protects the data **physically and logically**, whether at rest (storage) or in transit (network).

Key Points

Aspect	Explanation
Purpose	To secure data from unauthorized access by converting it into an unreadable form.
Visibility	Data is completely unreadable without the decryption key. Even database admins can't read it unless they have the key.
Reversibility	Reversible only with a key . Encryption is designed to be decrypted.
Example	- Original SSN: 123-45-6789- Encrypted (AES-256): U2FsdGVkX1+4nM0gE1yH1g==
Implementation	Implemented using algorithms like AES, RSA, or DES. Can be at rest (disk encryption) or in transit (HTTPS, VPN).
Use Cases	- Storing sensitive data (SSN, passwords, credit cards) securely- Sending secure messages or files over networks- Full disk encryption for servers

Types of Encryption

1. **Symmetric Encryption**
 - o Same key for encryption and decryption.
 - o Faster, used for large data volumes.
 - o Example: AES, DES.
 2. **Asymmetric Encryption**
 - o Uses **public/private key pair**.
 - o Slower, used for secure communication, digital signatures.
 - o Example: RSA, ECC.
-

Analogy

Think of **encryption** like **locking your data in a safe**: nobody can open it without the key, and the contents are physically secured.

3. Masking vs Encryption – Side-by-Side

Feature	Data Masking	Encryption
Purpose	Hide data visually	Protect data physically and logically

Feature	Data Masking	Encryption
Original Data	Exists in storage (sometimes altered in copies)	Converted to unreadable format
Reversibility	Usually irreversible (dynamic masking can be reversed for authorized users)	Reversible with key
Security Level	Prevents unauthorized viewing in apps/UI	Strong protection against unauthorized access and theft
Use Case	Test environments, limited UI exposure, compliance	Data at rest, data in transit, secure storage
Performance Impact	Low (especially static masking)	Higher (computational overhead of encryption/decryption)
Example	SSN: 123-45-6789 → XXX-XX-6789	SSN: 123-45-6789 → U2FsdGVkX1+4nM0gE1yH1g==

Key Takeaways

1. **Masking is for usability and limited visibility**, often in internal systems where some users can see only partial data.
2. **Encryption is for strong protection against theft or unauthorized access**, regardless of who views the data.
3. **Both are often used together**: Mask the data for developers/UI and encrypt it in storage for security.

If you want, I can also make a **diagram showing masking vs encryption visually**, which makes it super easy to remember.

Do you want me to do that?

Creating a **role in Snowflake** is straightforward, but you need the proper privileges (SECURITYADMIN or ACCOUNTADMIN) to do it. Here's a **detailed guide**:

1. Basic Syntax

```
CREATE ROLE <role_name>;
```

- `<role_name>`: Name of the new role you want to create.
- Example:

```
CREATE ROLE LIMITED_ACCESS_ROLE;
```

2. Grant the role to a user

After creating the role, you must **assign it to a user** so they can use it:

```
GRANT ROLE LIMITED_ACCESS_ROLE TO USER <username>;
```

- Example:

```
GRANT ROLE LIMITED_ACCESS_ROLE TO USER BHAASKAR;
```

3. Grant privileges to the role

Roles by themselves have **no privileges**. You must grant permissions for databases, schemas, tables, etc.

Example: Granting usage and select privileges

```
-- Allow the role to use a database
GRANT USAGE ON DATABASE MY_DB TO ROLE LIMITED_ACCESS_ROLE;

-- Allow the role to use a schema
GRANT USAGE ON SCHEMA MY_DB.MY_SCHEMA TO ROLE LIMITED_ACCESS_ROLE;

-- Allow the role to read data from a table
GRANT SELECT ON TABLE MY_DB.MY_SCHEMA.MY_TABLE TO ROLE LIMITED_ACCESS_ROLE;
```

4. Activate the role in your session

```
USE ROLE LIMITED_ACCESS_ROLE;
```

- After this, all operations are done with the privileges of `LIMITED_ACCESS_ROLE`.
-

5. Check your roles

- List all roles in your account:

```
SHOW ROLES;
```

- List roles assigned to a user:

```
SHOW GRANTS TO USER <username>;
```

6. Optional: Role hierarchy

- Roles can **inherit privileges** from other roles:

```
GRANT ROLE LIMITED_ACCESS_ROLE TO ROLE FULL_ACCESS_ROLE;
```

- Here, `FULL_ACCESS_ROLE` will also have all privileges of `LIMITED_ACCESS_ROLE`.
-

✔ Summary Steps to Create and Use a Role

1. Create the role:

```
CREATE ROLE LIMITED_ACCESS_ROLE;
```

2. Grant the role to a user:

```
GRANT ROLE LIMITED_ACCESS_ROLE TO USER BHAASKAR;
```

3. Grant privileges to the role:

```
GRANT USAGE ON DATABASE MY_DB TO ROLE LIMITED_ACCESS_ROLE;  
GRANT SELECT ON TABLE MY_DB.MY_SCHEMA.MY_TABLE TO ROLE LIMITED_ACCESS_ROLE;
```

4. Activate the role:

```
USE ROLE LIMITED_ACCESS_ROLE;
```

I can also make a **ready-to-use Snowflake script** that creates a role, assigns it to a user, and grants database/table privileges automatically.

Do you want me to create that?

That error means Snowflake doesn't know **which virtual warehouse** to use for running your queries.

A **warehouse** in Snowflake provides the compute resources needed to execute SQL statements like `GRANT`, `SELECT`, `INSERT`, etc.

✔ Fix: Select or create a warehouse

Option 1: Use an existing warehouse

If you already have one, simply activate it with:

```
USE WAREHOUSE COMPUTE_WH;
```

(Replace `COMPUTE_WH` with your actual warehouse name — for example, `DEV_WH`, `ANALYTICS_WH`, etc.)

Option 2: If no warehouse exists, create one

You can create a new warehouse if you have the right privileges:

```
CREATE WAREHOUSE COMPUTE_WH  
  WITH WAREHOUSE_SIZE = 'XSMALL'  
  AUTO_SUSPEND = 60  
  AUTO_RESUME = TRUE  
  INITIALLY_SUSPENDED = TRUE;  
  
USE WAREHOUSE COMPUTE_WH;
```

Option 3: Verify available warehouses

To see which warehouses exist and which you can use:

```
SHOW WAREHOUSES;
```

⚡ Example full flow

```
-- Step 1: Check available warehouses  
SHOW WAREHOUSES;  
  
-- Step 2: Choose one and activate it  
USE WAREHOUSE COMPUTE_WH;  
  
-- Step 3: Then run your GRANT statement again  
GRANT SELECT ON TABLE EMP_DB.HR_SCHEMA.EMPLOYEE TO ROLE READ_ONLY_ROLE;
```

Would you like me to show how to automatically select a warehouse every time you log in (so you don't get this error again)?

Perfect 🎯 — here's how you can **automatically set a default warehouse** every time you log in to Snowflake, so you never see the *"No active warehouse selected"* error again.

✔ Option 1: Set the Default Warehouse for Your User

You can assign a default warehouse to your **user profile** — this is the easiest and most reliable way.

```
-- Syntax
ALTER USER <username> SET DEFAULT_WAREHOUSE = <warehouse_name>;

-- Example
ALTER USER JOHN SET DEFAULT_WAREHOUSE = COMPUTE_WH;
```

After this, every time **JOHN** logs in, Snowflake will automatically use `COMPUTE_WH` as the active warehouse.

You can verify it:

```
SHOW USERS LIKE 'JOHN';
```

Look for the column **DEFAULT_WAREHOUSE**.

✔ Option 2: Set it in SnowSQL (CLI Users)

If you connect via the **SnowSQL command-line tool**, add this line in your `~/.snowsql/config` file:

```
[connections.myaccount]
accountname = your_account
username = JOHN
warehouse = COMPUTE_WH
role = FULL_ACCESS_ROLE
database = EMP_DB
schema = HR_SCHEMA
```

Then connect using:

```
snowsql -c myaccount
```

This automatically logs in with your preferred warehouse, database, schema, and role.

✔ Option 3: Set it in Snowflake Web UI Session

If you use the **Snowflake Web UI**, you can do it like this:

1. Go to **Snowsight (UI)** → top right corner → click your **profile icon** → **Preferences**.
2. Under **Default Warehouse**, choose your warehouse (e.g., `COMPUTE_WH`).
3. Click **Save**.

Now every new worksheet/session will start with that warehouse active.

✔ Option 4: Use a “Session Script” (for shared worksheets)

You can include this at the top of your worksheet or script:

```
USE ROLE FULL_ACCESS_ROLE;  
USE WAREHOUSE COMPUTE_WH;  
USE DATABASE EMP_DB;  
USE SCHEMA HR_SCHEMA;
```

That way, whenever you or teammates open the worksheet, the environment is ready to go.

Would you like me to show how to **automatically set both the default warehouse and default role** together for your user (so all sessions open with correct role + warehouse)?